

Ingeniería y Calidad de Software

Jake and Drosh

24-06-2025

Capítulo 16: Quality

El Capítulo 16 del libro “Succeeding with Agile” se titula “**Quality**” (**Calidad**) y aborda la importancia fundamental de la calidad en el desarrollo de software ágil, especialmente en el contexto de Scrum. Mike Cohn, el autor, relata su experiencia personal al inicio de su carrera donde se dio cuenta de la **responsabilidad individual de la calidad** en una startup, lo que luego transformó en una mentalidad de equipo.

1. Integrar las pruebas en el proceso

- **Problema tradicional:** Las pruebas ocurrían al final del ciclo de desarrollo, cuando ya era costoso y difícil corregir errores.
 - **Enfoque ágil:** Scrum exige que la calidad sea **parte central del desarrollo**, no una actividad separada.
 - **Buenas prácticas observables:** Incluyen programación en pares, inspecciones de código, pruebas unitarias (o TDD), refactorización continua, integración continua y **propiedad colectiva del código**.
 - **Indicador de calidad integrada:** La cantidad de errores detectados debe mantenerse constante durante el sprint, no concentrarse al final.
-

2. La pirámide de automatización de pruebas

Mike Cohn propone una jerarquía que guía **qué tipos de pruebas automatizar** y en qué proporción:

- **Base:** pruebas unitarias

- Son rápidas, baratas y precisas.
- Evalúan componentes individuales (métodos, clases).
- **Medio: pruebas de servicios o de integración**
 - Evalúan funcionalidades completas sin depender de la interfaz de usuario.
 - Más lentas que las unitarias, pero más estables que las de UI.
 - Herramientas sugeridas: FitNesse.
- **Cima: pruebas de interfaz de usuario (UI)**
 - Frágiles, lentas y costosas.
 - Se deben reducir al mínimo necesario.

Objetivo: Automatizar **lo antes posible**, idealmente dentro del mismo sprint donde se desarrolla la funcionalidad.

3. ATDD (Acceptance Test-Driven Development)

- Es una práctica que ayuda a **validar el comportamiento esperado** de una función antes de su implementación.
- Se basa en **Condiciones de Satisfacción (COS)**, que detallan los criterios de aceptación de cada historia de usuario.
- Permite a testers o analistas derivar pruebas de aceptación sin depender del Product Owner en todo momento.
- Los ejemplos, apoyados por conversaciones y texto explicativo, pueden convertirse en **pruebas automatizadas verificables**.

4. Gestión de la deuda técnica

- La **deuda técnica** es el costo de mantener código deficiente o no terminado.

- No toda deuda es negativa, pero debe **pagarse rápidamente** o se vuelve un riesgo.
 - Para la deuda técnica de pruebas manuales, se recomienda:
 1. **Detener la hemorragia:** Automatizar lo fácil de inmediato.
 2. **Mantenerse al día:** Automatizar las nuevas funcionalidades desde el inicio.
 3. **Ponerse al día:** Reducir gradualmente la deuda acumulada del pasado.
-

5. La calidad es un esfuerzo de equipo

- La responsabilidad por la calidad **no es solo del tester**, sino de **todo el equipo**.
 - Los beneficios son concretos: mayor productividad, menos errores y más tiempo para agregar valor.
 - Empresas como Salesforce vieron mejoras significativas al adoptar estas prácticas.
-

Conclusión

La calidad en Scrum no es una fase posterior, sino un **elemento continuo del desarrollo ágil**, sustentado por:

- **Buenas prácticas técnicas**
- **Automatización bien distribuida**
- **Cultura de responsabilidad compartida**
- **Gestión activa de la deuda técnica**

Mike Cohn destaca que **la calidad no se prueba, se construye desde el inicio**.

Un extra: Explicación acerca de ATDD

El **ATDD (Acceptance Test-Driven Development)** o **Desarrollo Guiado por Pruebas de Aceptación** es una práctica que busca **definir los criterios de aceptación antes de implementar una funcionalidad**, asegurando que el desarrollo cumpla exactamente con lo que el cliente espera.

A continuación te explico **cómo se hace o se logra el ATDD**, paso a paso:

1. Comenzar con una historia de usuario clara

Cada historia debe incluir:

- Una descripción del comportamiento esperado.
 - Sus **Condiciones de Satisfacción (COS)**: criterios que indican cuándo la historia puede considerarse completa.
-

2. Colaborar para definir ejemplos concretos

- Participan el **Product Owner**, desarrolladores y testers.
- A partir de las COS, se generan **ejemplos específicos** de entrada y salida.
- Estos ejemplos funcionan como **casos de prueba de aceptación**.

Ejemplo: Historia: “Como cliente, quiero ver un mensaje de error si ingreso una contraseña incorrecta.”

COS:

- Si la contraseña no coincide, se muestra “Contraseña incorrecta”.

Ejemplo concreto:

Dado que soy un usuario registrado,
Cuando ingreso una contraseña incorrecta,
Entonces debo ver un mensaje: "Contraseña incorrecta".

3. Convertir los ejemplos en pruebas automatizadas

- Se utilizan herramientas como:
 - **Cucumber**, **SpecFlow**, **Behave** (basadas en Gherkin).
 - **FitNesse**, para pruebas más tabulares.
- Los ejemplos se escriben en lenguaje natural estructurado (Given–When–Then) y se vinculan al código.

Esto genera **pruebas ejecutables**, que **fallan al principio** porque aún no se ha implementado la funcionalidad.

4. Desarrollar el código necesario para pasar la prueba

- El equipo implementa el código funcional que **satisface las condiciones definidas**.
 - Se repite el ciclo de prueba–código hasta que todas las pruebas de aceptación **pasan**.
-

5. Refactorizar si es necesario

- Una vez que las pruebas pasan, se puede mejorar el código sin temor a romper el comportamiento, ya que las pruebas actúan como una red de seguridad.
-

6. Revisar y mantener las pruebas

- Las pruebas de aceptación se mantienen junto con el código.
 - Sirven como documentación viva de cómo debe comportarse el sistema.
-

¿Qué se necesita para que funcione?

- **Colaboración constante** entre roles.
- Historias de usuario bien escritas con COS claras.

- Herramientas que soporten ATDD.
 - Cultura de equipo orientada a la calidad desde el inicio.
-

Resultado

- Se evita la ambigüedad de los requerimientos.
- Se construye solo lo necesario.
- Se mejora la comunicación con el cliente.
- Se obtienen pruebas automatizadas que validan el comportamiento real del sistema.